

GSAPから始める Webアニメーション入門

GreenSock Animation Platform で学ぶ Web アニメーション

本日のゴール

GSAP を使って Web アニメーションが作れるようになる！

最終的に、**観光PRサイト**をアニメーション付きで作ります。

もくじ

#	内容
1	GSAP とは？
2	基本の Tween — to / from / fromTo / set
3	プロパティ & イージング
4	Timeline — アニメーションの連結と制御
5	Stagger — 時間差アニメーション
6	コールバック & 制御メソッド
7	ScrollTrigger — スクロール連動
8	観光PRサイト作成
9	まとめ & 事例紹介

Chapter 1

GSAP とは？

GSAP (GreenSock Animation Platform) とは

- Web 上で最も広く使われるアニメーションライブラリ
- CSS アニメーション・ `requestAnimationFrame` のラッパーではなく、独自の高精度タイマーで動作
- 使用サイト例: Nike, Google, Microsoft, Apple...

 **2024年～ すべてのプラグインが無料に！**

Webflow による買収で、ScrollTrigger・SplitText・MorphSVG など
すべてのプラグインが無料で使えるようになりました。

なぜ GSAP を学ぶ？

比較項目	CSS Animation	Web Animations API	GSAP
学習コスト	低	中	中
制御性	低	中	高
順番に実行	困難	やや困難	簡単
ScrollTrigger	なし	なし	内蔵
ブラウザ互換	高	中	高

業界標準として広く使われている！

セットアップ

CDN (最も手軽)

```
<script src="https://cdn.jsdelivr.net/npm/gsap@3/dist/gsap.min.js"></script>
```

これだけで GSAP が使えるようになります！

デモ：セットアップ確認

`<iframe scrolling="no" title="GSAP セットアップ確認" src="https://codepen.io/mocaffy/embed/ByzeZwb?default-tab=html%2Cresult&editable=true" frameborder="no" loading="lazy" allowtransparency="true"> See the Pen`

[GSAP セットアップ確認](#)

on

[CodePen](#)

`. </iframe>`

 [CodePen](#) | `examples/01-basics/setup-basic.html`

Chapter 2

基本の Tween

Tween とは

Tween = 2 つの状態の間（between）を補間するアニメーションのこと

GSAP には **4 つの基本メソッド** がある：

メソッド	説明
<code>gsap.to()</code>	現在の状態 → 指定した状態 へ
<code>gsap.from()</code>	指定した状態 → 現在の状態 へ
<code>gsap.fromTo()</code>	開始値と終了値の両方を指定
<code>gsap.set()</code>	即座に値をセット（duration: 0 の to）

基本構文:

```
gsap.to(ターゲット, { プロパティ: 値, ... });
```

gsap.to() — 現在 → 目標値

現在の状態から目標値に向かってアニメーション

```
// .box を右に 300px 移動 (1秒かけて)  
gsap.to(".box", {  
  x: 300,  
  duration: 1,  
});
```

```
// 複数プロパティを同時にアニメーション  
gsap.to(".box", {  
  x: 200,  
  rotation: 360,  
  scale: 1.2,  
  borderRadius: "50%",  
  duration: 2,  
});
```

デモ： `gsap.to()`

`<iframe scrolling="no" title="gsap.to() - 複数プロパティ" src="https://codepen.io/mocaffy/embed/QwERgOL?default-tab=js%2Cresult&editable=true" frameborder="no" loading="lazy" allowtransparency="true"> See the Pen`

`gsap.to() - 複数プロパティ`

on

[CodePen](#)

`. </iframe>`

 [CodePen](#) | `examples/01-basics/gsap-to.html`

よく使うプロパティ

```
gsap.to(".box", {  
  // 位置  
  x: 100, // 水平移動  
  y: 50,  // 垂直移動  
  
  // サイズ・回転  
  scale: 1.5, // 拡大縮小  
  rotation: 360, // 回転 (度)  
  
  // 見た目  
  opacity: 0.5, // 透明度  
  autoAlpha: 0, // opacity + visibility を同時制御  
  backgroundColor: "#e74c3c",  
  
  // 時間  
  duration: 2, // 秒数  
  delay: 0.5, // 遅延  
});
```

gsap.from() — 指定値 → 現在

指定した状態から現在の状態に向かってアニメーション（逆方向）

```
// 左から飛んでくる演出
gsap.from(".box", {
  x: -300,
  opacity: 0,
  duration: 1,
});
```

よくある使い方：ページ読み込み時のフェードイン

```
gsap.from(".hero-title", {
  y: 50,
  opacity: 0,
  duration: 1,
  delay: 0.3,
});
```

`from()` は「初期状態を一時的に変更 → 元に戻す」イメージ

デモ： `gsap.from()`

<iframe scrolling="no" title="gsap.from()" src="https://codepen.io/mocaffy/embed/MYedoOK?default-tab=js%2Cresult&editable=true" frameborder="no" loading="lazy" allowtransparency="true"> See the Pen [gsap.from\(\)](#)

on

[CodePen](#)

. </iframe>

 [CodePen](#) | `examples/01-basics/gsap-from.html`

gsap.fromTo() — 開始値と終了値を両方指定

```
gsap.fromTo(
  ".box",
  { x: -200, opacity: 0 }, // from (開始値)
  { x: 200, opacity: 1, duration: 1 }, // to (終了値)
);
```

いつ使う？

- `to()` や `from()` では **現在の状態に依存** する
- `fromTo()` なら **開始と終了を完全にコントロール** できる
- 繰り返し実行しても常に同じ結果になる

デモ： `gsap.fromTo()`

<iframe scrolling="no" title="gsap.fromTo()" src="https://codepen.io/mocaffy/embed/azZrwVW?default-tab=js%2Cresult&editable=true" frameborder="no" loading="lazy" allowtransparency="true"> See the Pen [gsap.fromTo\(\)](https://codepen.io/mocaffy/embed/azZrwVW?default-tab=js%2Cresult&editable=true)

on

[CodePen](#)

. </iframe>

 [CodePen](#) | `examples/01-basics/fromto.html`

gsap.set() — 即座にセット

アニメーションなしで値をセットする (`duration: 0` の `to()` と同等)

```
// 初期位置を設定
gsap.set(".box", { x: 100, opacity: 0 });

// その後アニメーション
gsap.to(".box", { x: 300, opacity: 1, duration: 1 });
```

使いどころ

- アニメーション前の **初期状態の設定**
- 複数要素を **一括でリセット**
- **条件分岐** による即時切り替え

デモ： `gsap.set()`

<iframe scrolling="no" title="gsap.set()" src="https://codepen.io/mocaffy/embed/qENGjVP?default-tab=js%2Cresult&editable=true" frameborder="no" loading="lazy" allowtransparency="true"> See the Pen [gsap.set\(\)](#)

on

[CodePen](#)

. </iframe>

 [CodePen](#) | `examples/01-basics/set.html`

ターゲットの指定方法

GSAP では **CSS セレクタ**（文字列）で動かす要素を指定します。

HTML	セレクタ	ルール
<code><div class="box"></code>	<code>".box"</code>	class には <code>.</code> （ドット）を付ける
<code><div id="hero"></code>	<code>"#hero"</code>	id には <code>#</code> （シャープ）を付ける
<code><h1></code>	<code>"h1"</code>	タグ名 はそのまま

```
gsap.to(".box", { x: 100 }); // class="box" の要素を動かす
gsap.to("#hero", { opacity: 0 }); // id="hero" の要素を動かす
gsap.to("h1", { y: -20 }); // すべての <h1> を動かす
```



`document.querySelector()` と同じ書き方です

Chapter 3

プロパティ & イージング

duration / delay / repeat / yoyo

プロパティ	型	デフォルト	説明
duration	number	0.5	アニメーション時間（秒）
delay	number	0	開始までの待機時間（秒）
repeat	number	0	繰り返し回数（-1 で無限）
repeatDelay	number	0	繰り返し間の待機時間
yoyo	boolean	false	繰り返し時に逆再生する
ease	string	"power1.out"	イージング関数

```
// 無限ループ + 往復
gsap.to(".box", {
  x: 300,
  duration: 1,
  repeat: -1,
  yoyo: true,
  repeatDelay: 0.5,
});
```

デモ：duration / delay

<iframe scrolling="no" title="duration & delay" src="https://codepen.io/mocaffy/embed/emzaRyz?default-tab=js%2Cresult&editable=true" frameborder="no" loading="lazy" allowtransparency="true"> See the Pen [duration & delay](#)

on

[CodePen](#)

. </iframe>

 [CodePen](#) | `examples/02-properties/duration-delay.html`

デモ：repeat / yoyo

<iframe scrolling="no" title="repeat & yoyo" src="https://codepen.io/mocaffy/embed/ogLRwpp?default-tab=js%2Cresult&editable=true" frameborder="no" loading="lazy" allowtransparency="true"> See the Pen

[repeat & yoyo](#)

on

[CodePen](#)

. </iframe>

 [CodePen](#) | `examples/02-properties/repeat-yoyo.html`

イーasing (Easing)

アニメーションの加速・減速カーブを決める関数

```
ease: "power1.out"
      ~~~~~ ~~~
      種類   方向
```

3つの方向

方向	説明	イメージ
<code>.in</code>	ゆっくり始まる → 加速	 → 
<code>.out</code>	速く始まる → 減速	 → 
<code>.inOut</code>	ゆっくり → 速く → ゆっくり	 →  → 

デフォルトは `"power1.out"` (自然な減速)

デモ：イージング方向（in / out / inOut）

<iframe scrolling="no" title="Easing Directions" src="https://codepen.io/mocaffy/embed/ByzeZJb?default-tab=js%2Cresult&editable=true" frameborder="no" loading="lazy" allowtransparency="true"> See the Pen

[Easing Directions](#)

on

[CodePen](#)

. </iframe>

 [CodePen](#) | `examples/02-properties/easing-directions.html`

イージングの種類

ease	特徴	使いどころ
"none" / "linear"	等速	プログレスバー
"power1" ~ "power4"	加速/減速の強さ	UI 全般
"back"	行き過ぎて戻る	ボタン押下
"bounce"	バウンド	落下・着地
"elastic"	ゴムのような弾性	注目させたい要素
"circ"	円弧的な動き	回転系
"expo"	指数的な緩急	高速な切り替え
"steps(n)"	n ステップで切り替え	スプライトアニメ

💡 [Ease Visualizer](#) で確認しよう！

デモ：イーasing比較

<iframe scrolling="no" title="Easing Comparison" src="https://codepen.io/mocaffy/embed/vEKwZpr?default-tab=js%2Cresult&editable=true" frameborder="no" loading="lazy" allowtransparency="true"> See the Pen

[Easing Comparison](#)

on

[CodePen](#)

. </iframe>

 [CodePen](#) | `examples/02-properties/easing-comparison.html`

Elastic / Back の調整

() の中の数値で **動きの強さ** を変えられる

elastic — ゴムのような弾み

```
ease: "elastic.out(1, 0.3)";  
//  
//           ↑      ↑  
//       振れ幅の大きさ 振動の速さ (小さいほど速い)
```

```
ease: "elastic.out(2, 0.3)"; // 振れ幅 大  
ease: "elastic.out(1, 0.8)"; // ゆっくり振動
```

back — 行き過ぎて戻る

```
ease: "back.out(1.7)";  
//  
//           ↑  
//       行き過ぎる量 (大きいほど大きく行き過ぎる)
```

```
ease: "back.out(3)"; // もっと行き過ぎる
```

デモ：Elastic / Back パラメータ

`<iframe scrolling="no" title="Elastic & Back Parameters" src="https://codepen.io/mocaffy/embed/KwMLqQw?default-tab=js%2Cresult&editable=true" frameborder="no" loading="lazy" allowtransparency="true">` See the Pen

[Elastic & Back Parameters](#)

on

[CodePen](#)

`. </iframe>`

 [CodePen](#) | `examples/02-properties/elastic-params.html`

演習 1：基本の Tween

`examples/exercises/01-basic-tween.html` を開こう

1. `gsap.to()` で `.box` を **右に 400px** 移動させる (2秒)
2. `gsap.from()` で `.box` を **上から降ってくる** 演出にする (opacity も 0 → 1)
3. `gsap.fromTo()` で `.box` を **x: -200 から x: 400** まで移動させる
4. `gsap.set()` で **背景色を赤に変更** し、その後 `to()` で **回転 360°** させる

演習 1

<iframe scrolling="no" title="演習1: 基本のTween" src="https://codepen.io/mocaffy/embed/WbxBOKN?default-tab=js%2Cresult&editable=true" frameborder="no" loading="lazy" allowtransparency="true"> See the Pen

演習1: 基本のTween

on

CodePen

. </iframe>

 CodePen | examples/exercises/01-basic-tween.html

演習 2：プロパティ & イージング

`examples/exercises/02-easing.html` を開こう

1. パルスアニメーション: `.box` を `scale: 1.3` → 元に戻るを **無限ループ** (yoyo を使用)
2. イージング比較: 5 つの `.box` にそれぞれ異なる ease で `x: 400` へ移動させる
 - `linear`, `power2.out`, `bounce.out`, `elastic.out`, `back.out`
3. カード登場: ボタンクリックで `.card` が `scale: 0` → `scale: 1` で登場。最も自然に見える ease を探す

演習 2

```
<iframe scrolling="no" title="演習2: プロパティ & イーasing"
src="https://codepen.io/mocaffy/embed/MYedoBe?default-tab=js%2Cresult&editable=true" frameborder="no"
loading="lazy" allowtransparency="true"> See the Pen
```

演習2: プロパティ & イーasing

on

CodePen

```
. </iframe>
```

 CodePen | `examples/exercises/02-easing.html`



Chapter 4

Timeline

Timeline とは

複数のアニメーションを順番に並べて管理 できるコンテナ

Timeline を使わない場合（delay で管理）

```
gsap.to(".box1", { x: 100, duration: 1 });  
gsap.to(".box2", { x: 100, duration: 1, delay: 1 }); // ← 計算が必要  
gsap.to(".box3", { x: 100, duration: 1, delay: 2 }); // ← 計算が必要
```

Timeline を使う場合

```
const tl = gsap.timeline();  
  
tl.to(".box1", { x: 100, duration: 1 })  
  .to(".box2", { x: 100, duration: 1 }) // 自動で前のアニメの後に配置  
  .to(".box3", { x: 100, duration: 1 });
```

Timeline なら 順番の変更・挿入・削除が簡単

デモ：Timeline 基本

```
<iframe scrolling="no" title="Timeline 基本" src="https://codepen.io/mocaffy/embed/RNRmgQY?default-tab=js%2Cresult&editable=true" frameborder="no" loading="lazy" allowtransparency="true"> See the Pen
```

[Timeline 基本](#)

on

[CodePen](#)

```
. </iframe>
```

 [CodePen](#) | `examples/03-timeline/timeline-basic.html`

Position パラメータ

Timeline で アニメーションの開始位置を細かく制御 する

```
const tl = gsap.timeline();
tl.to(".a", { x: 100, duration: 1 }) // デフォルト: 前の直後
  .to(".b", { x: 100, duration: 1 }, "<") // 前と同時に開始
  .to(".c", { x: 100, duration: 1 }, "<0.5") // 前の開始0.5秒後
  .to(".d", { x: 100, duration: 1 }, "-=0.5") // 前の終了0.5秒前
  .to(".e", { x: 100, duration: 1 }, "+=1"); // 前の終了1秒後
```

指定	意味
"<"	直前の開始と同時
"<0.5"	直前の開始 + 0.5秒後
"-=0.5"	直前の終了 - 0.5秒（オーバーラップ）
"+=1"	直前の終了 + 1秒（間を空ける）
2	タイムラインの2秒地点（絶対位置）

デモ：Position パラメータ

```
<iframe scrolling="no" title="Position Parameters" src="https://codepen.io/mocaffy/embed/wBWbeyb?default-tab=js%2Cresult&editable=true" frameborder="no" loading="lazy" allowtransparency="true"> See the Pen
```

[Position Parameters](#)

on

[CodePen](#)

```
. </iframe>
```

 [CodePen](#) | `examples/03-timeline/position-params.html`

ラベル (Labels)

タイムライン上に名前付きのマーカー を設定できる

```
const tl = gsap.timeline();

tl.to(".box1", { x: 100, duration: 1 })
  .addLabel("middle") // ← ラベルを追加
  .to(".box2", { y: 100, duration: 1 })
  .to(".box3", { x: 100, duration: 1 }, "middle") // ← ラベル位置に配置
  .to(".box4", { y: 100, duration: 1 }, "middle+=0.5"); // ← ラベル + 0.5秒
```

ラベルの便利な使い方

```
// ラベル位置へジャンプ
tl.play("middle");

// ラベル位置から逆再生
tl.reverse("middle");
```

デモ：ラベル

<iframe scrolling="no" title="Labels (ラベル) " src="https://codepen.io/mocaffy/embed/bNeXLXY?default-tab=js%2Cresult&editable=true" frameborder="no" loading="lazy" allowtransparency="true"> See the Pen [Labels \(ラベル\)](#)

on

[CodePen](#)

. </iframe>

 [CodePen](#) | `examples/03-timeline/labels.html`

Timeline のデフォルト設定 & ループ

defaults で共通プロパティをまとめる

```
const tl = gsap.timeline({
  defaults: { duration: 0.5, ease: "power2.out" },
});

tl.to(".a", { x: 100 }) // duration, ease を省略できる
  .to(".b", { y: 100 })
  .to(".c", { x: 200 });
```

ループアニメーション

```
const tl = gsap.timeline({
  repeat: -1, // 無限ループ
  yoyo: true, // 往復
  repeatDelay: 0.5,
});

tl.to(".dot", { y: -20, stagger: 0.1 });
```

ローディングアニメーションに最適！

デモ： defaults & ループ

<iframe scrolling="no" title="Timeline defaults & ループ" src="https://codepen.io/mocaffy/embed/JoKgpgP?default-tab=js%2Cresult&editable=true" frameborder="no" loading="lazy" allowtransparency="true"> See the Pen

[Timeline defaults & ループ](#)

on

[CodePen](#)

. </iframe>

 [CodePen](#) | `examples/03-timeline/defaults-loop.html`

演習 3 : Timeline

`examples/exercises/03-timeline.html` を開こう

1. **Timeline で順番に移動**: 3 つのボックスを `.to()` で順番に右へ移動させる Timeline を作る
2. **Position パラメータでオーバーラップ**: 3 枚のカードを順番にフェードイン。2 枚目以降は `"-=0.3"` で前と重ねる
3. **ラベルでジャンプ**: Timeline にラベルを付けて、ボタンクリックでその位置にジャンプさせる

演習 3

```
<iframe scrolling="no" title="演習3: Timeline" src="https://codepen.io/mocaffy/embed/XJKwgBM?default-tab=js%2Cresult&editable=true" frameborder="no" loading="lazy" allowtransparency="true"> See the Pen
```

[演習3: Timeline](#)

on

[CodePen](#)

```
. </iframe>
```

 [CodePen](#) | `examples/exercises/03-timeline.html`

Chapter 5

Stagger

Stagger とは

複数要素のアニメーション開始を時間差で並べる 機能

```
gsap.to(".box", {  
  x: 300,  
  duration: 1,  
  stagger: 1, // 各要素 1 秒間隔  
});
```

each vs amount

プロパティ	説明
<code>stagger: 1</code> or <code>each: 1</code>	各要素間の間隔が 1 秒固定
<code>amount: 1</code>	全要素の合計が 1 秒 （要素数で割る）

デモ : Stagger `each` / `amount`

`<iframe scrolling="no" title="Stagger: each vs amount" src="https://codepen.io/mocaffy/embed/ZYONyrO?default-tab=js%2Cresult&editable=true" frameborder="no" loading="lazy" allowtransparency="true">` See the Pen

[Stagger: each vs amount](#)

on

[CodePen](#)

`. </iframe>`

 [CodePen](#) | `examples/02-properties/stagger-each-amount.html`

Stagger の from (開始位置)

```
stagger: { each: 0.1, from: "start" } // 先頭から (デフォルト)
stagger: { each: 0.1, from: "end" }   // 末尾から
stagger: { each: 0.1, from: "center" } // 中央から外へ
stagger: { each: 0.1, from: "edges" }  // 両端から中央へ
stagger: { each: 0.1, from: "random" } // ランダム
```

デモ : Stagger from

<iframe scrolling="no" title="Stagger: from" src="https://codepen.io/mocaffy/embed/azZrwqw?default-tab=js%2Cresult&editable=true" frameborder="no" loading="lazy" allowtransparency="true"> See the Pen

[Stagger: from](#)

on

[CodePen](#)

. </iframe>

 [CodePen](#) | `examples/02-properties/stagger-from.html`

Stagger の repeat / yoyo (波のドット)

各要素が **独立して** 繰り返し / 往復する

```
gsap.to(".dot", {  
  y: -30,  
  duration: 0.5,  
  stagger: {  
    each: 0.15,  
    repeat: -1, // 各要素が独立して無限ループ  
    yoyo: true, // 各要素が独立して往復  
  },  
});
```

これで「波のように動くドット」が簡単に作れる

デモ：Stagger の波アニメーション

<iframe scrolling="no" title="Stagger Wave" src="https://codepen.io/mocaffy/embed/qENGjxV?default-tab=js%2Cresult&editable=true" frameborder="no" loading="lazy" allowtransparency="true"> See the Pen

[Stagger Wave](#)

on

[CodePen](#)

. </iframe>

 [CodePen](#) | `examples/02-properties/stagger-wave.html`

Chapter 6

コールバック & 制御メソッド

コールバック

アニメーションのイベントに応じて関数を実行 できる

```
gsap.to(".box", {  
  x: 300,  
  duration: 2,  
  onStart: () => console.log("開始！"),  
  onUpdate: () => console.log("更新中..."),  
  onComplete: () => console.log("完了！"),  
});
```

コールバック	タイミング
onStart	アニメーション開始時（初回のみ）
onUpdate	フレームごと（毎フレーム実行）
onComplete	アニメーション完了時
onRepeat	各繰り返し完了時
onReverseComplete	逆再生が完了した時

💡 Timeline にも使えます（例: ローディング完了後にメイン表示）

デモ：コールバック

```
<iframe scrolling="no" title="Callbacks" src="https://codepen.io/mocaffy/embed/WbxBOze?default-tab=js%2Cresult&editable=true" frameborder="no" loading="lazy" allowtransparency="true"> See the Pen
```

[Callbacks](#)

on

[CodePen](#)

```
. </iframe>
```

 [CodePen](#) | `examples/03-timeline/callbacks.html`

制御メソッド

Tween や Timeline の 再生を自在にコントロール する

```
const tween = gsap.to(".box", { x: 300, duration: 2, paused: true });

tween.play(); // 再生
tween.pause(); // 一時停止
tween.reverse(); // 逆再生
tween.restart(); // 最初から再生
tween.seek(1); // 1秒地点にジャンプ
tween.progress(0.5); // 50% 地点にジャンプ
tween.timeScale(2); // 2倍速
tween.kill(); // 破棄
```

デモ：制御メソッド

```
<iframe scrolling="no" title="Control UI" src="https://codepen.io/mocaffy/embed/EayzXEm?default-tab=js%2Cresult&editable=true" frameborder="no" loading="lazy" allowtransparency="true"> See the Pen
```

[Control UI](#)

on

[CodePen](#)

```
. </iframe>
```

 [CodePen](#) | `examples/03-timeline/control-ui.html`

Chapter 7

ScrollTrigger

ScrollTrigger とは

スクロール位置に応じてアニメーションを発火・制御 するプラグイン

セットアップ

```
<!-- GSAP本体 -->
<script src="https://cdn.jsdelivr.net/npm/gsap@3/dist/gsap.min.js"></script>

<!-- ScrollTrigger プラグイン -->
<script src="https://cdn.jsdelivr.net/npm/gsap@3/dist/ScrollTrigger.min.js"></script>

<script>
  // プラグインを登録（重要！）
  gsap.registerPlugin(ScrollTrigger);
</script>
```

基本的な使い方

```
gsap.from(".box", {
  opacity: 0,
  y: 50,
  scrollTrigger: {
    trigger: ".box", // トリガー要素
    start: "top 80%", // 開始位置
    markers: true, // デバッグ用
  },
});
```

start / end の指定

start: 'トリガー要素の位置 画面上の位置'

例:

start: 'top 80%'	→ 要素の上端が画面の80%位置に来たら
start: 'top center'	→ 要素の上端が画面中央に来たら
start: 'center center'	→ 要素の中央が画面中央に来たら

デモ：ScrollTrigger 基本（フェードイン）

<iframe scrolling="no" title="ScrollTrigger - フェードイン" src="https://codepen.io/mocaffy/embed/JoKqJvr?default-tab=js%2Cresult&editable=true" frameborder="no" loading="lazy" allowtransparency="true"> See the Pen

[ScrollTrigger - フェードイン](#)

on

[CodePen](#)

. </iframe>

 [CodePen](#) | `examples/04-scrolltrigger/fade-in.html`

toggleActions — スクロール方向で動作を変える

```
scrollTrigger: {  
  trigger: '.box',  
  start: 'top 80%',  
  end: 'bottom 20%',  
  toggleActions: 'play none none reverse'  
  //           ↗         ↑         ↑         ↖  
  //           onEnter  onLeave  onEnterBack onLeaveBack  
}
```

値	動作
play	再生
pause	一時停止
resume	再開
reverse	逆再生
none	何もしない

💡 よく使うパターン: "play none none reverse" (画面外に出たら元に戻る)

デモ： toggleActions

<iframe scrolling="no" title="ScrollTrigger - toggleActions" src="https://codepen.io/mocaffy/embed/wBWbejb?default-tab=js%2Cresult&editable=true" frameborder="no" loading="lazy" allowtransparency="true"> See the Pen

[ScrollTrigger - toggleActions](#)

on

[CodePen](#)

. </iframe>

 [CodePen](#) | `examples/04-scrolltrigger/toggle-actions.html`

markers でデバッグ

markers

```
scrollTrigger: {  
  trigger: '.box',  
  start: 'top 80%',  
  markers: true    // 緑 (start) と赤 (end) のラインが表示  
}
```

本番では必ず `false` にするか削除すること

scrub — スクロール量に連動

```
gsap.to(".box", {  
  x: 500,  
  rotation: 360,  
  scrollTrigger: {  
    trigger: ".section",  
    start: "top center",  
    end: "bottom center",  
    scrub: true, // スクロールに完全連動  
  },  
});
```

scrub の値	動作
true	スクロール位置に即座に追従
1	1秒かけてスムーズに追従

デモ： scrub

<iframe scrolling="no" title="ScrollTrigger - scrub" src="https://codepen.io/mocaffy/embed/LEZoLme?default-tab=js%2Cresult&editable=true" frameborder="no" loading="lazy" allowtransparency="true"> See the Pen

[ScrollTrigger - scrub](#)

on

[CodePen](#)

. </iframe>

 [CodePen](#) | `examples/04-scrolltrigger/scrub.html`

pin — 要素を固定する

```
gsap.to(".box", {  
  rotation: 360,  
  scrollTrigger: {  
    trigger: ".section",  
    start: "top top",  
    end: "+=200%", // スクロール量（画面2個分）  
    pin: true, // セクションを固定！  
    scrub: true,  
  },  
});
```

- `pin: true` でトリガー要素を画面に固定
- スクロールしても要素は動かず、アニメーションだけ進む
- 横スクロールや複雑な演出の基盤になる重要な機能

デモ：pin

<iframe scrolling="no" title="ScrollTrigger - pin" src="https://codepen.io/mocaffy/embed/gbMJRzZ?default-tab=js%2Cresult&editable=true" frameborder="no" loading="lazy" allowtransparency="true"> See the Pen

[ScrollTrigger - pin](#)

on

[CodePen](#)

. </iframe>

 [CodePen](#) | `examples/04-scrolltrigger/pin.html`

横スクロール (pin + scrub の応用)

```
const panels = document.querySelector(".panels");
const scrollAmount = panels.scrollHeight - window.innerWidth;

gsap.to(".panels", {
  x: () => -scrollAmount, // 関数形式で値を返す
  ease: "none",
  scrollTrigger: {
    trigger: ".panels-wrapper",
    start: "top top",
    end: () => "+=" + scrollAmount,
    pin: true, // 固定して
    scrub: 1, // スクロールに追従
    invalidateOnRefresh: true, // リサイズ時に再計算
  },
});
```

デモ：横スクロール

<iframe scrolling="no" title="ScrollTrigger - 横スクロール" src="https://codepen.io/mocaffy/embed/MYedoXe?default-tab=js%2Cresult&editable=true" frameborder="no" loading="lazy" allowtransparency="true"> See the Pen

[ScrollTrigger - 横スクロール](#)

on

[CodePen](#)

. </iframe>

 [CodePen](#) | `examples/04-scrolltrigger/horizontal-scroll.html`

Timeline + ScrollTrigger

```
const tl = gsap.timeline({
  scrollTrigger: {
    trigger: ".section",
    start: "top center",
    end: "bottom center",
    scrub: 1,
    markers: true,
  },
});

tl.from(".title", { y: 30, opacity: 0 })
  .from(".text", { y: 20, opacity: 0 }, "-=0.4")
  .from(".box", { scale: 0, stagger: 0.15 }, "-=0.2");
```

- Timeline のオプションに `scrollTrigger` を追加するだけ！
- スクロールで発火し、中のアニメーションが順番に実行される

デモ : Timeline + ScrollTrigger

`<iframe scrolling="no" title="Timeline + ScrollTrigger" src="https://codepen.io/mocaffy/embed/QwERgxW?default-tab=js%2Cresult&editable=true" frameborder="no" loading="lazy" allowtransparency="true">` See the Pen

[Timeline + ScrollTrigger](#)

on

[CodePen](#)

`. </iframe>`

 [CodePen](#) | `examples/04-scrolltrigger/timeline-scroll.html`

演習 4 : ScrollTrigger

`examples/exercises/04-scrolltrigger.html` を開こう

1. **フェードイン**: セクションが画面に入ったらボックスがフェードイン (`trigger`, `start`, `markers`)
2. **scrub**: スクロール量に連動してボックスが移動&回転 (`scrub: true`)
3. **Timeline + ScrollTrigger**: タイトル → テキスト → ボックスが `"-=0.4"` で重なりながら順番に登場

演習 4

<iframe scrolling="no" title="演習4: ScrollTrigger" src="https://codepen.io/mocaffy/embed/dPXRje?default-tab=js%2Cresult&editable=true" frameborder="no" loading="lazy" allowtransparency="true"> See the Pen

演習4: ScrollTrigger

on

CodePen

. </iframe>

 CodePen | examples/exercises/04-scrolltrigger.html



Chapter 8

観光PRサイト作成

作るもの

自分の出身地（都道府県・市区町村）のPRサイト

StackBlitz のテンプレートをベースに、
あなたの地元をPRするサイトを作ってみよう！

 [StackBlitz を開く](#)

 `examples/05-fukui-pr/index.html`

デモ：観光PRサイト

```
<iframe src="https://stackblitz.com/edit/stackblitz-starters-b9hpbykz?embed=1&file=index.html"
frameborder="no" loading="lazy" allowfullscreen> </iframe>
```

 [StackBlitz](https://stackblitz.com/edit/stackblitz-starters-b9hpbykz?embed=1&file=index.html) | `examples/05-fukui-pr/index.html`

サンプルサイトの構成

福井県PRサイトで使われている GSAP の技術：

セクション	使用技術
ローディング	Timeline + stagger
ヒーロー	Timeline（順番に登場）
イントロ	ScrollTrigger + カウントアップ
観光スポット	パララックス効果（scrub）
グルメ	横スクロール（pin + scrub）
アクセス	Timeline + ScrollTrigger

カスタマイズの手順

Step 1: テーマを決める

- どの地域をPRする？（都道府県 or 市区町村）
- どんな魅力を伝えたい？

Step 2: 必要な情報を集める

- 観光スポット（3つ程度）
- 名物・グルメ
- アクセス情報
- 画像（[Unsplash](#) などで探す）

カスタマイズのポイント

配色の変更（CSS カスタムプロパティ）

```
:root {  
  --color-primary: #1a5f4a; /* メインカラー */  
  --color-secondary: #d4a373; /* サブカラー */  
}
```

テキストの変更

```
<h1 class="hero-title">  
  <span class="hero-title-line">FUKUI</span>  
  <!-- ↑ 地域名 -->  
</h1>
```

画像の変更

```

```

⚠ 「Unsplash+」 マークの無い画像を選びましょう

やってみよう

必須: コンテンツを自分の地元に

1. 地域名・テキスト・画像を変更
2. 配色を変更 (CSS カスタムプロパティで最低3色)

選択: 好きなアニメーションをカスタマイズ ✨

(2つ以上やってみよう)

- **ease**: ヒーローの登場を `back.out` や `elastic.out` に変えてみる
- **stagger**: ローディング文字の間隔や `from` を変える
- **Timeline**: ヒーローの `"-=0.6"` を変えてタイミングを調整
- **scrub**: 横スクロールの追従速度を `0.5` → `2` に変える
- **パララックス**: 観光スポットの `y` 移動量を変える

チャレンジ ★

- オリジナルのセクション追加 + アニメーション

作業時間

約40分

- 10分：テーマ決め & 情報収集
- 20分：コードの編集
- 10分：グループ内で発表

困ったらスタッフに聞いてね！

発表タイム

作ったサイトをみんなに見せよう！

- どの地域を選んだ？
- こだわりポイントは？
- 使った GSAP の技術は？

Chapter 9

まとめ & ベストプラクティス

今日学んだこと

✓ GSAP の基本

- `gsap.to` / `gsap.from` / `gsap.fromTo` / `gsap.set`
- プロパティ (x, y, scale, rotation, opacity...)
- イージング (ease)

✓ Timeline & Stagger

- 複数アニメーションの制御 / タイミングの調整 (`<`, `+=`, `-=`)
- ループアニメーション (repeat, yoyo)
- Stagger (時間差アニメーション)

✓ コールバック & ScrollTrigger

- onStart / onComplete / onUpdate
- スクロール連動 (scrub) / ピン留め (pin) / 横スクロール
- Timeline + ScrollTrigger の組み合わせ

パフォーマンスのコツ

✓ 推奨: Transform プロパティを使う

```
// GPU アクセラレーションが効く（高速）  
gsap.to(".box", { x: 100, y: 50, scale: 1.5, rotation: 45 });  
// autoAlpha で opacity + visibility を同時制御  
gsap.to(".box", { autoAlpha: 0 });
```

✗ 避ける: レイアウトを変更するプロパティ

```
// リフロー（再計算）が発生する（低速）  
gsap.to(".box", { width: "200px", top: "100px" });
```

不要な Tween は kill()

```
ScrollTrigger.getAll().forEach((st) => st.kill());  
gsap.killTweensOf(".box");
```

デモ： autoAlpha パターン

<iframe scrolling="no" title="opacity vs autoAlpha" src="https://codepen.io/mocaffy/embed/GgqaEBP?default-tab=js%2Cresult&editable=true" frameborder="no" loading="lazy" allowtransparency="true"> See the Pen

[opacity vs autoAlpha](#)

on

[CodePen](#)

. </iframe>

 [CodePen](#) | `examples/patterns/autoalpha.html`

gsap.utils & gsap.matchMedia

ユーティリティメソッド

```
gsap.utils.random(0, 500); // ランダムな値
gsap.utils.random(["red", "blue"]); // 配列からランダム選択
gsap.utils.toArray(".box"); // NodeList を配列に変換
gsap.utils.clamp(0, 100, 150); // → 100 (範囲制限)
```

レスポンス対応

```
const mm = gsap.matchMedia();

mm.add("(min-width: 768px)", () => {
  // デスクトップ用アニメーション
  gsap.to(".sidebar", { x: 0, duration: 0.5 });
});

mm.add("(max-width: 767px)", () => {
  // モバイル用アニメーション
  gsap.to(".sidebar", { y: 0, duration: 0.5 });
});
```

GSAP 学習ロードマップ

ステップ	内容	重要度
1	<code>to()</code> / <code>from()</code> / <code>fromTo()</code> / <code>set()</code>	★★★★
2	<code>duration</code> / <code>delay</code> / <code>repeat</code> / <code>yoyo</code>	★★★★
3	Easing の使い分け	★★★★
4	Timeline + Position パラメータ	★★★★
5	Stagger	★★★☆
6	コールバック + 制御メソッド	★★★☆
7	ScrollTrigger	★★★★
8	<code>matchMedia</code> / <code>context</code>	★★★☆
9	SplitText / MotionPath 等のプラグイン	★☆☆☆

さらに学びたい人へ

- [GSAP 公式ドキュメント](#)
- [Ease Visualizer](#)
- [GSAP Cheat Sheet](#)
- [ScrollTrigger デモ](#)
- [CodePen で作例を見る](#)

実際に作って試すのが一番！

よくあるパターン集

ページロードアニメーション

```
const tl = gsap.timeline();
tl.from("nav", { y: -100, duration: 0.5 })
  .from(".hero h1", { opacity: 0, y: 50 }, "-=0.2")
  .from(".hero .cta", { opacity: 0, scale: 0.8 }, "-=0.2");
```

テキストカウンター

```
const counter = { value: 0 };
gsap.to(counter, {
  value: 12345,
  duration: 2,
  onUpdate: () => {
    el.textContent = Math.round(counter.value).toLocaleString();
  },
});
```

 [CodePen](#) | [examples/patterns/page-load.html](#) • [CodePen](#) | [examples/patterns/hover-card.html](#) • [CodePen](#) | [examples/patterns/text-counter.html](#)

弊社の事例紹介

事例1

- プロジェクト名: ふわっち
- URL: <https://whowatch.tv/s/snack/>
- 使用技術: GSAP, ScrollTrigger

事例2

- プロジェクト名: VTuber登竜門
- URL: <https://www.vmon.jp/>
- 使用技術: GSAP, ScrollTrigger

質疑応答

技術的な質問でも
会社についての質問でも
何でもどうぞ！